



ಜಿ. ಎಂ. ವಿಶ್ವವಿದ್ಯಾಲಯ
GM UNIVERSITY

P. B. Road, Davanagere – 577 006 KARNATAKA | INDIA

Faculty Name: Mr. Rajashekhar G C

Date:13-06-2026

Faculty of Computing IT
Master in Computer Applications
Second Semester
Assignment-1

Course: Python Programming (PC25CA5202)

Assignment Title: Smart Banking Fraud Detection and Financial Analytics System

Institution Name: GM University Davanagere

Department: Faculty of Computing and IT

Course: MCA

Semester: 2nd Semester

Subject Name: Python Programming

Subject Code: PC25CA5202

Project Title: Smart Banking Fraud Detection and Financial Analytics System

Academic Year: 2025-2026

Student Name: Preetham V R

USN: P25C01CA034

Project Guide

Guide Name: Mr. Rajashekhar G C

Department :Director FCIT , GM University

REVIEW – 1 : Problem Analysis & Research

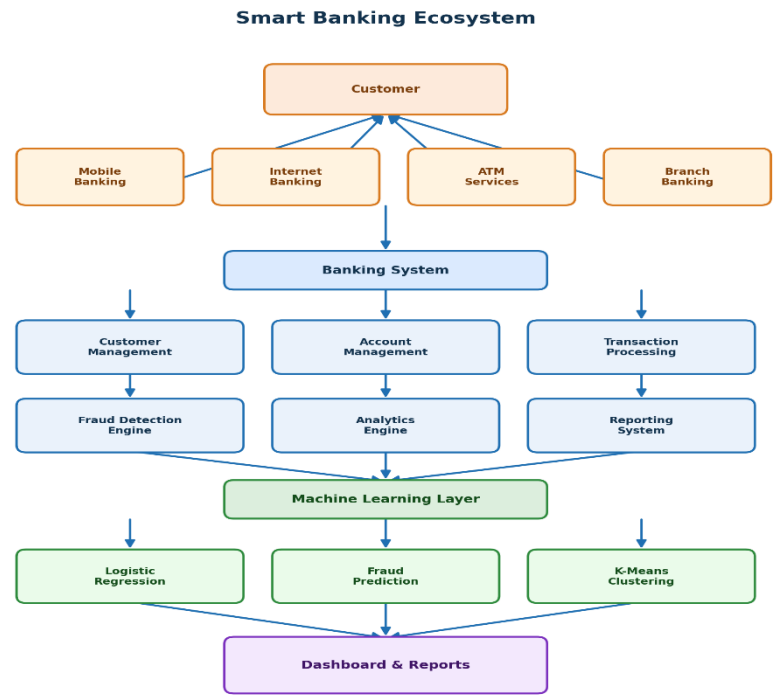
SMART BANKING FRAUD DETECTION AND FINANCIAL ANALYTICS SYSTEM

1. Introduction

The rapid growth of digital banking, mobile payments, internet banking, UPI transactions, and online financial services has significantly increased transaction volumes across banking institutions. While these technologies improve convenience and accessibility, they also create opportunities for fraudulent activities such as unauthorized transactions, account takeovers, money laundering, identity theft, and suspicious fund transfers.

Traditional fraud detection systems primarily rely on manual monitoring and static rule-based approaches, which often fail to identify emerging fraud patterns in real time. As transaction volumes increase, banks require intelligent systems capable of processing large-scale transaction data, identifying suspicious behaviour, and generating actionable insights.

Banking Ecosystem Diagram



Smart Banking Ecosystem: channels, core modules, ML layer, and dashboard

Advantages

- Real-time detection of suspicious transactions
- Reduced financial losses due to fraud
- Improved customer trust and account security
- Faster generation of financial reports
- Data-driven decision making for bank operations
- Automated risk categorization of customers

As a result of the absence of such a system:

- Fraudulent transactions go undetected for longer periods.
- Banks incur direct financial losses and reputational damage.
- Customer spending behaviour remains unanalyzed.
- Manual auditing increases operational cost and delay.

Therefore, an intelligent banking analytics platform is required that can process transactions in real time, detect fraud using machine learning, and provide financial insights to support decision-making.

2. Problem Statement

Modern banking systems process millions of financial transactions daily. Detecting fraudulent activities manually is difficult, expensive, and time-consuming. Fraudsters continuously develop new methods to bypass traditional security mechanisms.

Therefore, there is a need for an intelligent banking analytics platform capable of processing transactions, identifying suspicious activities, predicting fraudulent behaviour, and generating real-time financial insights.

2.1 The Cold-Start Challenge

New customers often have limited transaction history, which affects the reliability of behaviour-based fraud models.

Challenges:

- Insufficient behavioural data for new accounts.
- Difficulty identifying suspicious patterns without history.
- Limited fraud-prediction accuracy during onboarding.

Impact: Machine learning models struggle to classify risk accurately during the initial stages of customer activity. The system mitigates this by combining rule-based checks (amount thresholds, odd-hour activity) with the learned model, so new accounts are still screened from day one.

2.2 Operational Analytics Deficits

Many banking systems focus only on transaction processing and store data purely for record-keeping.

Challenges:

- Lack of customer spending analysis.
- Limited fraud trend monitoring.
- Delayed reporting.
- Poor decision support for bank officers.

Impact: Reduced operational efficiency and increased fraud exposure. The Analytics and Visualization modules in this project convert raw transaction logs into deposit/withdrawal totals, fraud ratios, and spending leaderboards automatically.

2.3 Scalability and Real-Time Performance Constraints

Modern banking systems process millions of transactions.

Challenges:

- High transaction volume.

- Real-time processing requirements.
- Large-scale fraud monitoring.
- Computational overhead of repeated model scoring.

Impact: Traditional systems fail to maintain performance and scalability. This project addresses the concern at a smaller scale using efficient Python data structures — a dictionary for $O(1)$ account lookup, a queue for FIFO transaction intake, and a stack for LIFO history retrieval — combined with a lightweight Logistic Regression model that scores a transaction in milliseconds.

3. Literature Survey

The following papers were reviewed to understand existing approaches to financial fraud detection and to justify the choice of Logistic Regression and K-Means Clustering for this project.

Paper Title	Authors	Year	Method	Findings
Financial Fraud Detection Based on Machine Learning: A Systematic Literature Review	Abdulalem Ali et al.	2022	ML Survey	Identified SVM and ANN as widely used fraud detection techniques.
Credit Card Fraud Detection in the Era of Disruptive Technologies	Ahmed Manjoum et al.	2023	Systematic Review	Highlighted deep learning and feature engineering approaches.
Financial Fraud Detection Applying Data Mining Techniques	K. G. Al-Hashedi, P. Magalingam	2021	Data Mining Review	Reviewed 34 fraud detection techniques across domains.

Survey on Credit Card Fraud Detection Techniques	P. R. Shimpi, V. Kadroli	2015	ML Survey	Compared classical fraud detection algorithms.
Real-Time Transaction Fraud Detection via Machine Learning Techniques	K. D. Tripathi, V. S. Rajput	2020	Review	Emphasized real-time detection mechanisms.
Credit Card Fraud Detection Using Machine Learning: A Study	Pooja Tiwari et al.	2021	Comparative Study	Compared Logistic Regression, Random Forest, SVM, and Neural Networks.
Credit Card Fraud Detection Using Machine Learning: A Survey	Y. Lucas, J. Jurgovsky	2020	Survey	Addressed class imbalance and concept drift in fraud data.
A Comprehensive Survey of Fraud Detection Methods in Credit Card Systems	Raghad H. Alwan	2022	Data Mining Survey	Reviewed hybrid rule-based and ML fraud detection models.

Summary of Literature Survey:

Most studies conclude that a hybrid approach — combining rule-based checks with a supervised classifier — offers a good balance of accuracy and interpretability for banking use cases, which is the approach followed in this project (rule engine + Logistic Regression + K-Means).

4. Feature Schema

Feature Name	Data Type	Description	Role in ML Pipeline
Transaction_ID	String	Unique transaction identifier (UUID-based).	Record tracking
Account_Number	String	Customer account number.	Identification
Amount	Float	Transaction value in ₹.	Numeric feature
Transaction_Type	Categorical	Deposit / Withdraw / Transfer.	Encoded feature
Hour	Integer (0–23)	Hour of day the transaction occurred.	Numeric feature
Location	Categorical	City / International.	Encoded feature
Device_Type	Categorical	Mobile / Web / Tablet / ATM / POS.	Encoded feature
Previous_Balance	Float	Account balance before the transaction.	Numeric feature
Current_Balance	Float	Account balance after the transaction.	Numeric feature
Fraud	Integer	0 = Legitimate, 1 = Fraudulent.	Target label (Y)

5. Dataset Collection

A synthetic dataset of 12,000 transaction records was generated using `dataset_generator.py`, simulating realistic banking activity across 8 cities (including an International category), 5 device types, and 3 transaction types. Fraud labels were injected using three weighted rules — high-value transactions, odd-hour activity (12 AM–5 AM), and international transfers — plus 1% random noise to emulate real-world unpredictability.

Metric	Value
Total transaction records	12,000
Unique customer accounts	6,658
Deposit transactions	3,925
Withdrawal transactions	4,023
Transfer transactions	4,052
Legitimate transactions (fraud = 0)	6,385
Fraudulent transactions (fraud = 1)	5,615
Total deposit value	₹19,78,78,587.84
Total withdrawal value	₹20,02,93,290.91
Total transfer value	₹20,23,14,836.64

Dataset Importance:

- Training the Logistic Regression fraud classifier.
- Training the K-Means clustering model for customer risk segmentation.
- Populating the SQLite database (banking.db) with realistic customers, accounts, and transactions.
- Feeding the Analytics and Visualization modules for dashboard reporting.

Dataset:

transaction_id	account_number	amount	transaction_type	hour	location	device_type	previous_balance	current_balance	fraud
7d36fa60	4042	15777.58	transfer	0	Pune	Web	41612.13	25834.55	1

aff936ac	9046	7964 6.07	withdraw	7	Kolkata	ATM	49922.14	- 29723.93	0
2f912808	3288	5811 1.46	transfer	12	Bangalore	ATM	171090.98	112979.52	1
7b5f4af1	1030	8697 .43	transfer	18	International	Tablet	76429.57	67732.14	1
ab44a436	6204	7066 .94	withdraw	17	International	POS	44252.21	37185.27	0
58c10cf9	5946	5650 2.50	deposit	5	Pune	Tablet	187500.93	244003.43	1
ec80f0c6	5179	5915 9.11	deposit	0	Chennai	Moblie	188135.96	247295.07	1
d4cf6435	4707	5929 5.60	deposit	18	Kolkata	POS	165122.07	224417.67	1
3811fadc	5278	8953 3.11	withdraw	14	Kolkata	Web	193480.81	103947.70	1
bcf4d929	6518	3669 7.46	deposit	15	Chennai	Moblie	12275.49	48972.95	0
6016d840	5717	9684 3.18	deposit	4	Kolkata	Moblie	46923.10	143766.28	1
6f9a9822	8613	4443 7.84	transfer	11	Delhi	Tablet	92331.90	47894.06	0
2f58cc4e	1797	6338 7.44	transfer	14	Bangalore	Tablet	36234.56	- 27152.88	1
e658cd88	5115	2870 .23	transfer	2	Hyderabad	ATM	167773.44	164903.21	1

4e3217 7f	8504	2989 6.10	transfer	3	Hyder abad	ATM	173558.5 1	143662. 41	1
ef0ca68 f	9005	3537 7.49	deposit	19	Pune	Mobil e	4031.55	39409.0 4	0
666f3f7 9	4949	5814 8.27	transfer	0	Pune	Mobil e	111457.1 1	53308.8 4	1
f8a6ce5 1	5138	5727 6.22	transfer	23	Pune	Tablet	143734.7 0	86458.4 8	0
040e7f1 5	8709	9101 7.52	transfer	18	Intern ational	ATM	170630.1 3	79612.6 1	1

6. Fraud Detection Approach

6.1 Rule-Based Detection

Before the ML model runs, each transaction is screened against fixed rules implemented in `fraud_detection.py`. A risk score is accumulated and the transaction is flagged if the total crosses a threshold of 70:

- Rule 1 — Amount > ₹50,000 → +50 risk points.
- Rule 2 — Transaction between 12 AM and 5 AM → +40 risk points.
- Rule 3 — International transfer → +30 risk points.

6.2 Machine Learning Detection — Logistic Regression

Logistic Regression is used to classify a transaction as legitimate (0) or fraudulent (1) based on six numeric/encoded features: amount, hour, location, device type, previous balance, and current balance. The model estimates the probability of fraud using the sigmoid function:

$$P(\text{fraud} = 1 | X) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n)}}$$

Logistic Regression fraud-probability function

Where β_0 is the intercept, $\beta_1 \dots \beta_n$ are the learned coefficients for each feature ($x_1 \dots x_n$), and the output is thresholded at 0.5 to produce the final 0/1 fraud label.

6.3 Model Evaluation

The saved model (models/fraud_model.pkl) was evaluated on a held-out 20% test split (2,400 transactions) of the dataset:

Metric	Score
Accuracy	73.9%
Precision	70.8%
Recall	73.6%
F1-Score	72.2%

Confusion Matrix (rows = actual, columns = predicted):

	Predicted: Legitimate	Predicted: Fraud
Actual: Legitimate	960	336
Actual: Fraud	291	813

6.4 K-Means Clustering — Customer Risk Segmentation

In addition to per-transaction classification, K-Means Clustering ($k = 3$) is trained on the same feature set to group transactions/customers into unsupervised risk bands, independent of the labelled fraud outcome:

- Cluster 0 — Low Risk
- Cluster 1 — Medium Risk
- Cluster 2 — High Risk

This lets the dashboard flag customers whose overall transaction pattern is unusual even if no single transaction crosses the rule thresholds.

7. Requirement Specification

Functional Requirements

- Customer registration, update, view, and deletion.
- Account creation, balance view, deposit, withdrawal, and transfer.
- Transaction processing through a FIFO queue with full history logging.
- Rule-based and ML-based fraud detection on every transaction.
- K-Means based customer risk clustering.
- Analytics dashboard: totals, fraud alerts, top spenders.
- PDF financial and fraud report generation.

Non-Functional Requirements

- Reliability
- Security (encrypted-at-rest customer data goals)
- Scalability
- Maintainability
- Availability
- Performance
- Accuracy

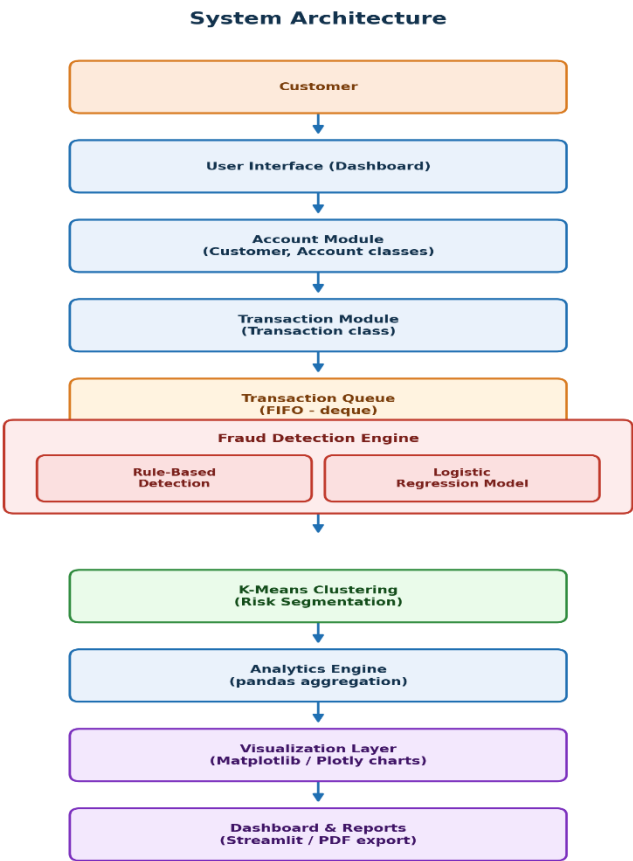
Hardware Requirements

Component	Specification
Processor	Intel i5 or higher
RAM	8 GB minimum
Storage	50 GB free space
Internet	Required for dataset/library setup

Software Requirements

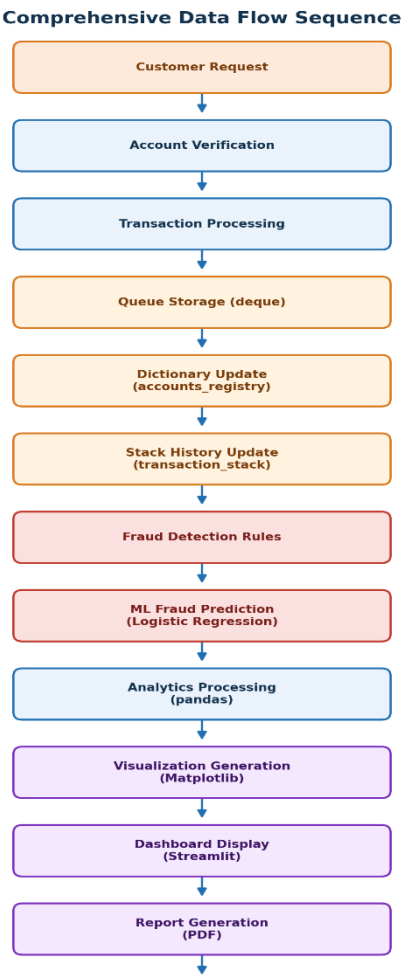
Software	Purpose
Python 3.12+	Core programming language
Pandas / NumPy	Data processing
Scikit-Learn	Logistic Regression & K-Means models
Matplotlib / Plotly	Data visualization
Streamlit	Interactive dashboard
SQLite (banking.db)	Persistent data storage
FPDF	PDF report generation
VS Code	Development environment

8. System Architecture



End-to-end system architecture from customer request to dashboard/report output

9. Comprehensive Data Flow Sequence



Data flow from customer request through fraud scoring to reporting

10. Data Structures Used

Data Structure	Used In	Purpose
Dictionary	bank.py accounts_registry	Maps account number → Account object for O(1) lookup.
Queue (deque)	transaction.py transaction_queue	FIFO buffer for incoming transaction requests before processing.

Stack (list)	transaction.py transaction_stack	—	LIFO store of processed transactions for fast recent-history retrieval.
--------------	-------------------------------------	---	--

11. Object-Oriented Design

BankEntity is defined as an abstract base class (using Python's abc module) with abstract methods display() and validate(). Customer implements these directly, and Account inherits from Customer — extending it with balance, account_type, deposit(), withdraw(), and transfer() — demonstrating both inheritance and abstraction in a single hierarchy:

- BankEntity (Abstract) → display(), validate()
- Customer(BankEntity) → customer_id, name, email, phone
- Account(Customer) → account_number, balance, account_type, deposit(), withdraw(), transfer()
- Transaction → transaction_id, amount, transaction_type, process_transaction(), execute()

12.Future Machine Learning Implementation

Algorithm 1: Logistic Regression

Logistic Regression is a supervised machine learning algorithm used to classify a transaction as Legitimate or Fraudulent based on patterns learned from historical labelled data.

Working:

Step 1

Collect past transactions, each already labelled as Legitimate (0) or Fraudulent (1).

↓

Step 2

Extract features from each transaction — amount, hour of day, location, device type, previous balance, and current balance.

↓

Step 3

Train the model to learn the weighted relationship between these features and the fraud label.



Step 4

For a new transaction, the trained model calculates the probability of it being fraudulent using the sigmoid function.



Step 5

If the probability crosses the decision threshold (0.5), the transaction is classified as Fraudulent; otherwise, it is classified as Legitimate.

Example

Transaction

Amount = ₹65,000, Hour = 3 AM, Location = International, Device = Web

The model computes a high fraud probability (0.82) because the amount is large, the hour falls in the high-risk window, and the location is International.

Therefore

Since 0.82 exceeds the 0.5 threshold, the transaction is classified as Fraudulent and flagged for review.

Used for:

- Classifying every incoming transaction as Normal or Fraud
- Assigning a fraud probability score for prioritizing review

Algorithm 2: K-Means Clustering

K-Means Clustering is an unsupervised machine learning algorithm used to group transactions or customers into risk clusters, without relying on the fraud label — useful for catching unusual patterns a classifier might miss.

Working:

Step 1

Take the same transaction features (amount, hour, location, device type, previous balance, current balance).



Step 2

Choose $K = 3$ cluster centres representing Low, Medium, and High Risk groups.



Step 3

Assign each transaction to its nearest cluster centre based on Euclidean distance.



Step 4

Recalculate each cluster centre as the average of its assigned transactions, and repeat until the clusters stabilize.



Step 5

Label each customer's overall behaviour according to the risk cluster they fall into.

Example

Customer

Frequently transacts at odd hours with high amounts from International locations.

This customer's transactions consistently land in the "High Risk" cluster, even if no single transaction alone crosses the fraud threshold.

Therefore

The customer is flagged for closer monitoring based on their overall pattern, not just one transaction.

Used for:

- Identifying unusual customer behaviour patterns
- Segmenting customers into Low / Medium / High risk groups

13. Results Summary

- 12,000-row synthetic transaction dataset generated and stored in dataset/transactions.csv.
- SQLite database (banking.db) populated with 6,658 customers and 6,658 linked accounts.
- Logistic Regression model trained and saved to models/fraud_model.pkl — 73.9% test accuracy.
- K-Means clustering model trained and saved to models/cluster_model.pkl for risk segmentation.
- analytics.py successfully aggregates deposits, withdrawals, transaction counts, and fraud alerts on demand.
- visualization.py generates all four charts above directly from live database data.
- report_generator.py compiles the statistics and charts into a downloadable PDF (reports/Banking_Report.pdf) using FPDF.
- dashboard.py provides an interactive Streamlit interface for bank officers to monitor alerts in real time.

14. Future Enhancements

- Deep learning-based fraud detection (e.g., LSTM/GNN for sequential and relational patterns).
- Blockchain-backed transaction integrity.
- Real-time streaming fraud alerts.
- Mobile banking integration.
- AI-based financial advisory assistant.
- Voice-based banking assistant.

15. Conclusion

The Smart Banking Fraud Detection and Financial Analytics System provides a complete banking solution by combining transaction management, fraud detection, machine learning, clustering, and financial analytics into a single intelligent platform. The project demonstrates practical implementation of Data Structures (dictionary, queue, stack), Object-Oriented Programming (inheritance, abstraction), Machine Learning (Logistic Regression, K-Means Clustering), and Data Visualization, making it suitable for academic evaluation and as a foundation for real-world fraud monitoring applications.

References

- [1] Abdulalem Ali et al., "Financial Fraud Detection Based on Machine Learning: A Systematic Literature Review", Applied Sciences, 2022.
- [2] Khaled Gubran Al-Hashedi and Pritheega Magalingam, "Financial Fraud Detection Applying Data Mining Techniques", Computer Science Review, 2021.
- [3] Ahmed Manjoum et al., "Credit Card Fraud Detection in the Era of Disruptive Technologies", Journal of King Saud University, 2023.
- [4] Priya Ravindra Shimpi and Vijayalaxmi Kadroli, "Survey on Credit Card Fraud Detection Techniques", IJECS, 2015.
- [5] Kapil Dev Tripathi and Vikas Singh Rajput, "Real-Time Transaction Fraud Detection via Machine Learning Techniques", IJCSE, 2020.
- [6] Pooja Tiwari et al., "Credit Card Fraud Detection Using Machine Learning: A Study", 2021.
- [7] Yvan Lucas and Johannes Jurgovsky, "Credit Card Fraud Detection Using Machine Learning: A Survey", 2020.
- [8] Raghad H. Alwan, "A Comprehensive Survey of Fraud Detection Methods in Credit Card Systems", 2022.